# Parser Generators

## Concepts of Programming Languages

CAS CS 320

# Announcements

- Assignment 5 due tonight at 8pm

- No assignment to be released today; study

- No lecture next Tuesday (Monday’s schedule)

- Midterm 1 next Thursday during lecture

- Students in A2 section (2pm – 3:15pm) can give exam in A1 section (3:30pm – 4:45pm); see Piazza

# Example

```
<expr> ::= x
         | if <expr> then <expr>
         | if <expr> then <expr> else <expr>
```

# What can we do about ambiguity?

# Fixity

The fixity of an operator refers to where an operator is placed relative to its arguments:

```
prefix      f x , (- x)

postfix     a!

infix       a * b, a + b, a mod b

mixfix      if b then x else y
```

# Not Great Solution: Polish Notation

```
- / + 2 * 1 - 2 3
```

**is equivalent to**

```
- (2 + (1 * (- 2) / 3)
```



To avoid ambiguity, we can make all operators prefix (or postfix) operators. *We don't even need parentheses*

*(This how early calculators worked)*

# Not Great Solution: Lots of Parentheses

```
<expr>  ::= (<op1> <expr>)
          | (<expr> <op2> <expr>)
          | <var>
<op1>   ::= not
<op2>   ::= and | or
<var>   ::= x | y | z
```

```
(((not x) and (not (not y)) or z)

((((x or y) or z) or x) or y)

(not ((not x) and (not y)) or
      (x and z))

(x and y)
```

If we want infix operators, we *could* add parentheses around all operators

But we run into a similar issue: *Too many parentheses are difficult to read*

Can we get away without,
or with fewer parentheses?

# Two Properties of Infix operators

**Associativity:** *How should arguments be grouped in an expression like* `1 + 2 + 3 + 4`*?*

**Precedence:** *How should arguments be grouped in an expression like* `1 + 2 * 3 + 4`*?*

# Associativity

The **associativity** of an infix operator refers to how its arguments are grouped in the absence of parentheses:

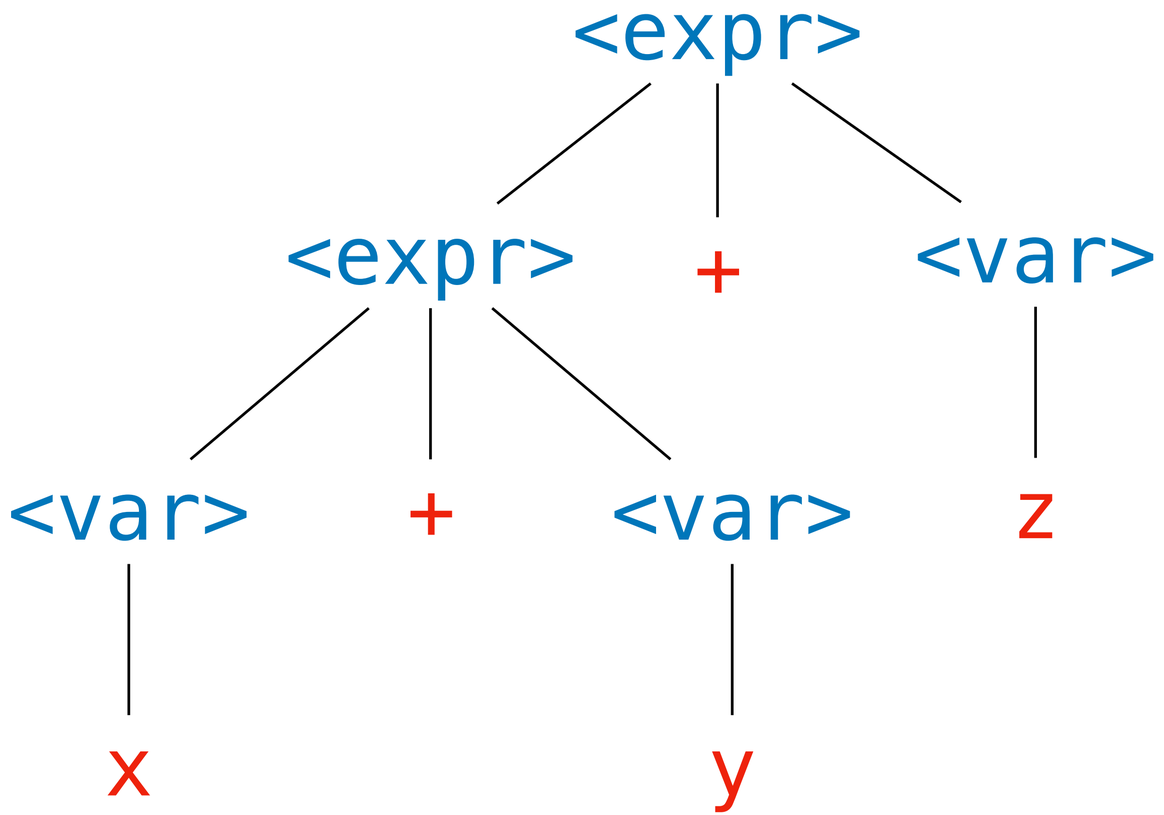
```
left associative     1 +  2 +  3 ⇒ (1 +  2) +  3

right associative    a -> b -> c ⇒  a -> (b -> c)
```

# Associativity

```
<expr> ::= <expr> + <expr>
         | <var>
<var>  ::= x | y | z
```

x + y + z ⇒




"*add the sum of x and y to z*"

***How do we enforce that we get a tree of this shape?***

# The Culprit

```
<expr> ::= <expr> + <expr>
         | <var>
<var>  ::= x | y | z
```

Any time we have a rule like this, we should be suspicious...

<expr> + <expr> ⇒ <expr> + <expr> + <expr>

**Which <expr> did we replace?**

# One Solution: Breaking Symmetry

```
<expr> ::= <expr> + <var>
         | <var>
<var>  ::= x | y | z
```

```
<expr>
<expr> + <var>
<expr> + z
<expr> + <var> + z
<expr> + y + z
<var> + y + z
x + y + z
```

*We make sure that one of the arguments must be "simpler"*

By enforcing that the second argument is a <var>, we'll get the left-associative parse tree

# And For Right Associativity

```
<type> ::= <base> -> <type>
         | <base>
<base> ::= ()
```

```
<type>
<base> -> <type>
() -> <type>
() -> <base> -> <type>
() -> () -> <type>
() -> () -> <base>
() -> () -> ()
```



() → base

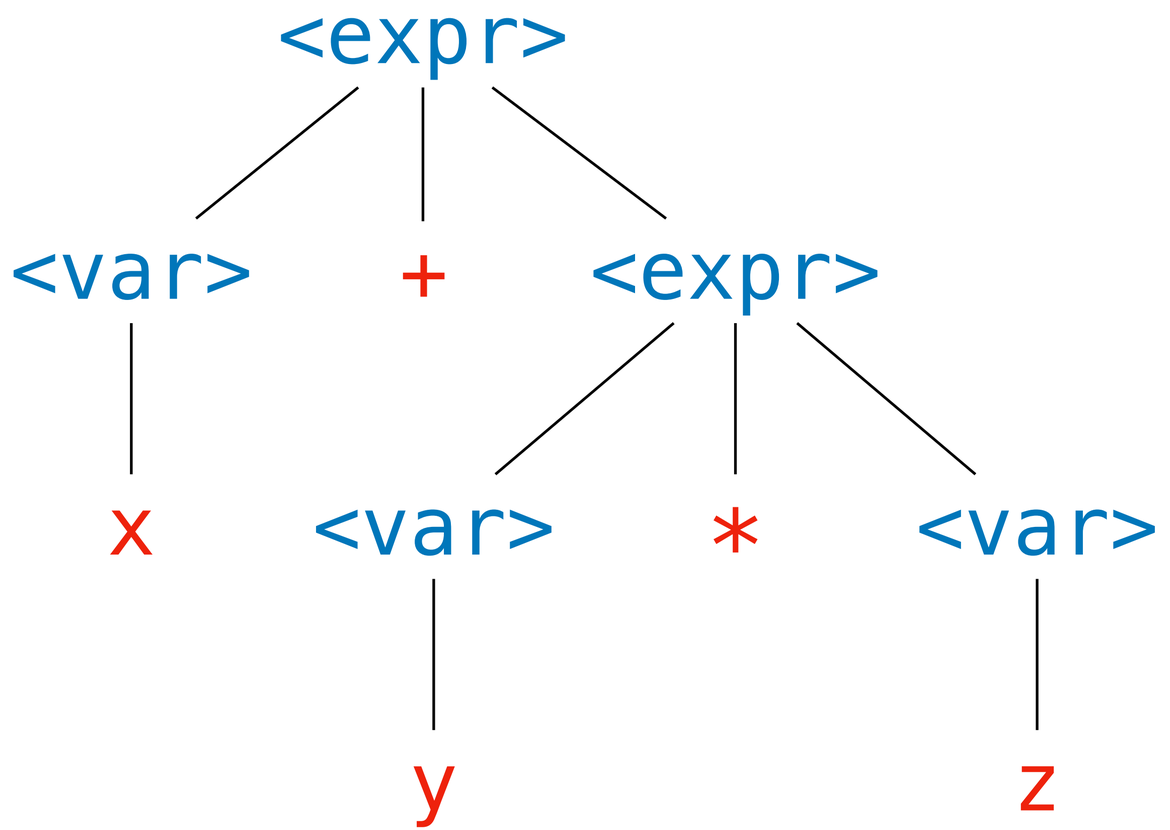
() → ()

For right associativity, we break symmetry by "factoring out" the *left* argument

# Multiple Operators

```
<expr> ::= <expr> <op> <var>
         | <var>
<op>   ::= + | *
<var>  ::= x | y | z
```

**x + y * z** ⇒




"*add x to the product of y and z*"

***Question.*** *What if we have multiple operators? Which one should "bind tighter"?*

# Precedence

$$2 + 3 \times 6 = 2 + (3 \times 6) = 20$$

The precedence of an operator refers to order in which an operator should be considered, relative to other operators

*Example.* **PEMDAS (paren, exp, mul, div, add, sub)**

Higher precedence means it "binds tighter"

# Dealing with Precedence within the Grammar

```
<expr> ::= <expr> + <term>
         | <term>
<term> ::= <term> * <var>
         | <var>
<var>  ::= x | y | z
```

We factor out the * part of the <expr> rule

Note that we handle *lower* precedence terms first, since terms *deeper* in the parse tree are evaluated first

# Example

```
<expr> ::= <expr> + <term>
         | <term>
<term> ::= <term> * <var>
         | <pars> <var>
<pars> ::= <var>
<var>  ::= x | y | z
```

**x + y * z**

⟨expr⟩

⟨expr⟩ + ⟨term⟩

⟨term⟩ + ⟨term⟩

⟨var⟩ + ⟨term⟩

x + ⟨term⟩

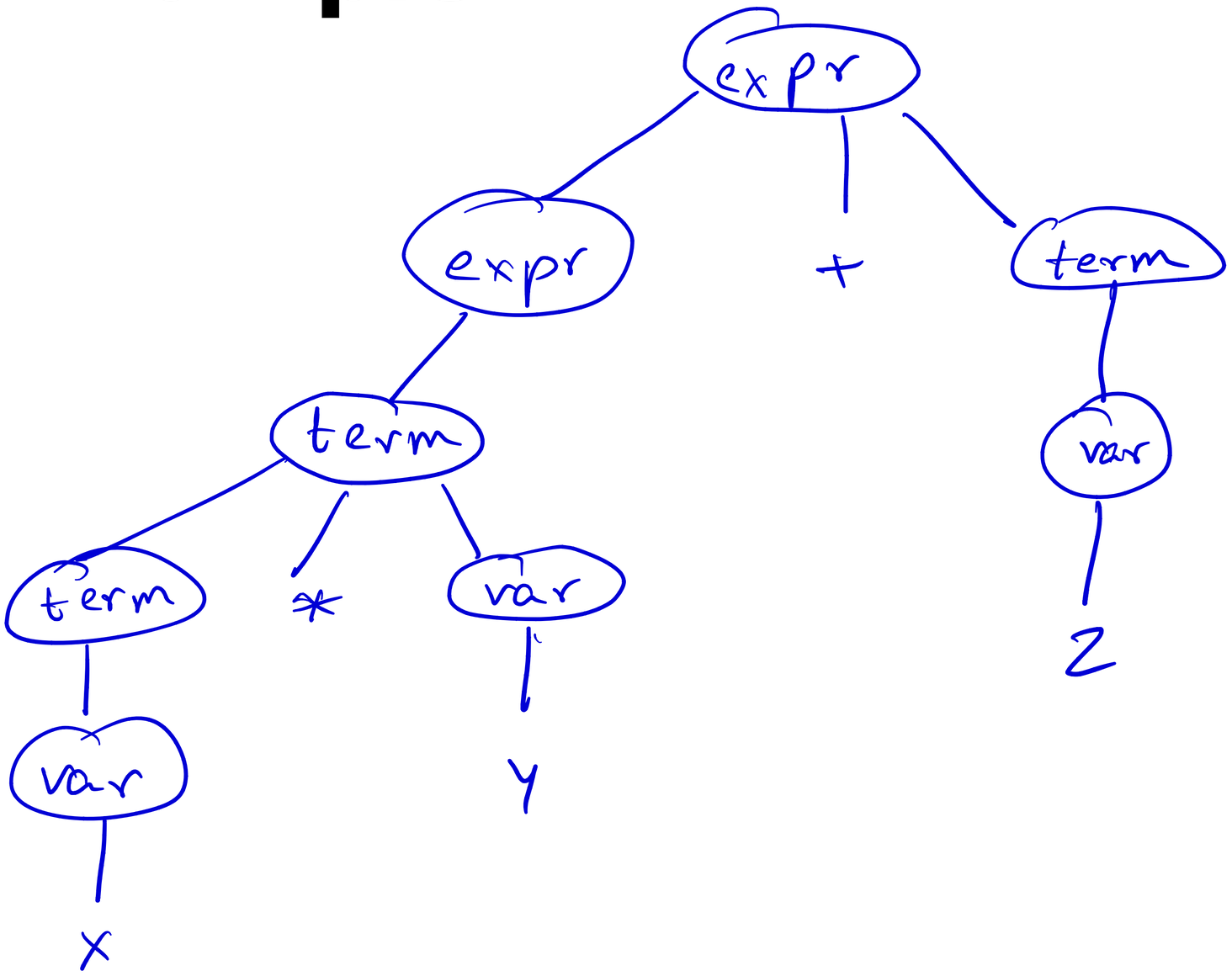
x + ⟨term⟩ * ⟨var⟩

x + ⟨var⟩ * ⟨var⟩

x + y * ⟨var⟩

x + y * z

# Example




```
<expr> ::= <expr> + <term>
         | <term>
<term> ::= <term> * <var>
         | <pars>
<pars> ::= ( <expr> )
<var>  ::= x | y | z
```

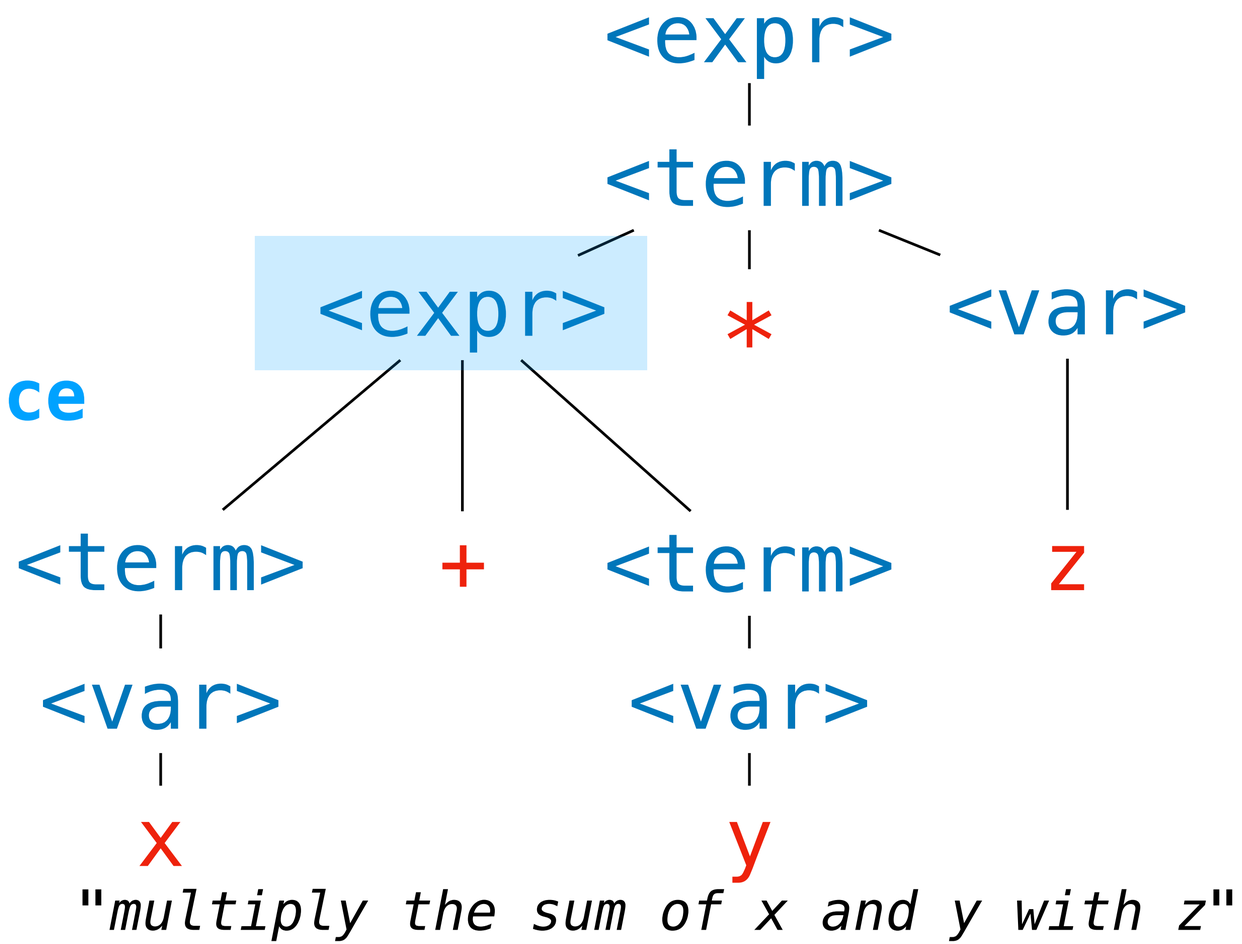
**x * y + z**

# The Issue of Parentheses Returns

```
<expr> ::= <expr> + <term>
         | <term>
<term> ::= <term> * <var>
         | <var>
<var>  ::= x | y | z
```

**No, we need to introduce parentheses again**

**( x + y ) * z**




***Question.*** *Can we derive this parse tree?*

# Dealing with Parentheses

```
<expr> ::= <expr> + <term>
         | <term>
<term> ::= <term> * <var>
         | <pars>
<pars> ::= <var> | ( <expr> )
<var>  ::= x | y | z
```

We further factor out the part of the rule for parentheses. Note that any expression can appear in the parentheses

*(This is a circular, or mutually recursive, production rule)*

# Outline

- Extend our BNF syntax to be a bit more convenient
- Introduce **parser generators**
- Demo **Menhir,** the parser generator for this course

# Extended BNF

# Extended BNF

```
<expr>           ::= <only-mul-div> { (+ | -) <only-mul-div> }
<only-mul-div>   ::= <var-or-parens> { (* | /) <var-or-parens> }
<var-or-parens>  ::= x | ( <expr> )
```

Extended BNF is syntactic sugar. **It is no more expressive than BNF**

But it allows us to specify:

- Optional parts of production rule
- Repeated parts of a production rule

**Note: EBNF means different things to different people**

# Optional Syntax

**BNF:**

```
<expr> ::= if <expr> then <expr>
         | if <expr> then <expr> else <expr>
```

**EBNF:**

```
<expr> ::= if <expr> then <expr> [ else <expr> ]
```

**Menhir:**

```
expr:
  | IF; e1 = expr; THEN; e2 = expr; e3_opt = else?
    { match e3_opt with
      | None -> It (e1, e2)
      | Some e3 -> Ite (e1, e2, e3)
    }
else:
  | ELSE; e = expr { e }
```

# Repetition Syntax

**BNF:**

```
<expr>  ::= [ ] | [ <elems> ]
<elems> ::= <expr> | <expr> ; <elems>
```

**EBNF:**

```
<expr> ::= [ ] | [ <expr> { <elem> } ]
<elem> ::= ; <expr>
```

**Menhir:**

```
elem:
  | SEMICOLON; e = expr { e }
expr:
  | LBRAK; RBRAK { List_lit [] }
  | LBRAK; e = expr; es = elem*; RBRAK
    { List_lit (e :: es) }
```

# Interlude: Regular Expressions

# Regular Expressions (Formally)

**Regular expressions (Regex)** provide a compact way of describing regular grammars:

- A **terminal symbols** is a regex
- **[ t1 ... tk ]** is a regex describing an any one of the symbols **t1**, **t2**, ..., **tk**
- **( e1 | ... | ek)** is a regex describing any one of the expressions **e1**, **e2**, ..., **ek**
- **exp*** is a regex describing zero or more occurrences of **exp**
- **exp+** is a regex describing one or more occurrences of **exp**
- **exp?** is a regex describing zero or one occurrences of **exp**

# Example

```
<s> ::= a <s>
<s> ::= b <a>
<a> ::= ϵ
<a> ::= c <a>
```

*is equivalent to*

```
a*bc*
```

*or*

```
'a'* 'b' 'c'*
```

*in ocamllex syntax*

# Example: Numbers and Variables

```
-?[0-9]+
```

***numbers***

```
[a-z_][a-z0-9A-Z_']*
```

***variables***

We'll leave it there, take CS332 if you want more, or read the Wikipedia page...

# Parsing with Menhir

# General Parsing

***In Theory.*** *Determine if a given sentence is recognized by a given grammar*

***In Practice.*** *Given a grammar, write a program which converts a string recognized by that grammar into an ADT*

# Today

```
<prog>  ::= <expr>

<expr>  ::= let <var> = <expr> in <expr>
          | <expr1>

<bop>   ::= + | - | * | /

<expr1> ::= <expr1> <bop> <expr1>
          | <num>
          | <var>
          | ( <expr> )

<num>   ::= 0 ; DUMMY VALUE
<var>   ::= x ; DUMMY VALUE

; In lex.mll:
;
; let num = '-'? ['0'-'9']+
; let var = ['a'-'z' '_'] ['a'-'z' 'A'-'Z' '0'-'9' '_' '\'']*
```

Operators in order of increasing precedence:

| Operator | Associativity |
|---|---|
| +, - | left |
| *, / | left |

We'll be building a parser for this grammar

# A Note on Associativity and Precedence

```
%token PLUS
%token MINUS
%token TIMES
%token DIVIDE

%left PLUS, MINUS
%left TIMES, DIVIDE
```

In Menhir, we actually won't deal with associativity and precedence using new non-terminals

Using a parser generator we'll often be able to *specify* the precedence of an operator

# A Rough Sketch

1. Specify the tokens (i.e., terminal symbols) of the grammar

2. Specify the rules of the grammar (using a BNF-like syntax)

3. Specify the rules of the lexer (i.e., which strings go to which tokens)